

MPECI 2025-2026

Simulação baseada em Eventos Discretos

Geração de Valores Aleatórios

Análise de Resultados de Simulação

Simulação baseada em eventos discretos

Considere-se um sistema:

- representado por um conjunto de variáveis de estado;
- o estado do sistema pode mudar quando ocorre um evento (de um conjunto de tipos possíveis de eventos);
- os instantes de tempo de ocorrência dos eventos são variáveis aleatórias.

Simulação baseada em eventos discretos: modelação da evolução temporal de um sistema cujo estado só pode mudar nos instantes de ocorrência dos eventos.

- Quando um evento (de um determinado tipo) acontece, o estado do sistema pode mudar.
- Entre dois eventos consecutivos, o estado do sistema permanece constante pelo que após o instante de um evento a simulação pode avançar para o instante do próximo evento.

Objetivos da simulação de eventos discretos

Visualização do comportamento de um sistema:

- neste caso, pretende-se visualizar a evolução temporal de alguns (ou todos os) valores das variáveis de estado do sistema

Estimação de parâmetros de desempenho:

- por cada parâmetro de desempenho de interesse, é também necessário considerar outras variáveis, designadas por *contadores estatísticos*
- os *contadores estatísticos* armazenam ao longo da simulação informação que permite no fim da simulação determinar os parâmetros de desempenho de interesse

Elementos de um simulador de eventos discretos

Variáveis de estado:

- descrevem o estado do sistema em cada instante de tempo

Contadores estatísticos (se houver):

- variáveis que armazenam informação estatística relativa aos parâmetros (de interesse) de desempenho do sistema

Eventos:

- tipos de ocorrências e, para cada tipo, de que forma alteram as variáveis de estado e/ou os contadores estatísticos

Relógio de simulação:

- variável que indica, em cada instante, o instante de tempo de simulação (tempo de simulação $\neq$ tempo de computação)

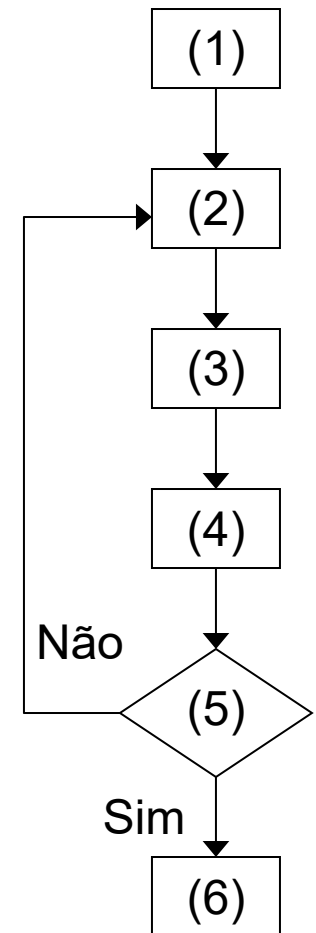
Lista de eventos:

- lista onde são armazenados os instantes de ocorrência dos eventos futuros (e possíveis parâmetros associados)

Outros variáveis auxiliares (se necessário)

Fluxograma de um simulador de eventos discretos

- (1) Inicializa as variáveis de estado, os contadores estatísticos e a lista de eventos com o(s) primeiro(s) evento(s)
- (2) Determina qual o próximo evento da lista de eventos
- (3) Avança o relógio de simulação para o instante de ocorrência desse evento
- (4) Executa as ações associadas a esse evento (geração de novos eventos, atualização das variáveis de estado e/ou dos contadores estatísticos)
- (5) Determina se a simulação deve terminar; se não, retorna ao passo (2)
- (6) Atualiza (se necessário) os contadores estatísticos e determina os parâmetros de desempenho



Exemplo 1:

Simulação de um servidor de *video-streaming*

Considere-se um servidor de *vídeo-streaming* caracterizado por:

- os pedidos de filmes chegam segundo um processo de Poisson a uma taxa de λ pedidos por minuto
- os filmes têm uma duração exponencialmente distribuída com média de $1/\mu$ (em minutos)
- cada filme é transmitido a um débito de B (em Mbps)
- o servidor tem uma capacidade para servir até M filmes em simultâneo

Pretende-se estimar:

- Probabilidade de bloqueio (percentagem de pedidos de filmes que são recusados porque o servidor está a transmitir filmes à sua capacidade máxima)
- Débito binário médio transmitido pelo servidor

Exemplo 1:

Simulação de um servidor de *video-streaming*

Eventos:

ARRIVAL – pedido de um filme

DEPARTURE – fim da transmissão de um filme

Variáveis de estado:

STATE – número de filmes em transmissão

Critério de paragem:

N – a simulação termina no fim da transmissão do Nésimo filme *

* requer a variável auxiliar TRANSMITTED com o número de filmes transmitidos até ao instante presente

Exemplo 1:

Simulação de um servidor de *video-streaming*

Contadores estatísticos necessários:

N_ARRIVALS – no. de pedidos de filmes desde o início até ao instante presente

BLOCKED – número de pedidos de filmes recusados desde o início até ao instante presente

LOAD – integral do débito binário transmitido pelo servidor desde o início até ao instante presente

Cálculo final dos parâmetros de desempenho:

Probabilidade de bloqueio (PB):

$$PB = \text{BLOCKED} / \text{N_ARRIVALS}$$

Débito binário médio transmitido pelo servidor (DB):

$$DB = \text{LOAD} / \text{tempo total simulado}$$


```
function VideoStreamingSimulator(lambda, invmiu, B, M, N)
```

```
% Events:
```

```
ARRIVAL= 0;           % request of a movie  
DEPARTURE= 1;         % end of a movie transmission
```

```
% Initialization of variables and List of Events:
```

```
Clock= 0;             % simulation clock  
STATE= 0;             % no. of movies in transmission  
TRANSMITTED= 0;       % no. of transmitted movies  
EventList= [ARRIVAL, Clock + exprnd(1/lambda)];
```

```
while TRANSMITTED < N
```

```
    EventList= sortrows(EventList,2);    % sort EventList by time  
    event= EventList(1,1);               % register event type in first row  
    Clock= EventList(1,2);               % update current clock with time instant of first row  
    EventList(1,:)= [];                  % delete first row of EventList
```

```
    switch event
```

```
        case ARRIVAL
```

```
            EventList= [EventList; ARRIVAL, Clock + exprnd(1/lambda)];    % add future event
```

```
            if STATE < M
```

```
                STATE= STATE + 1;
```

```
                EventList= [EventList; DEPARTURE, Clock + exprnd(invmiu)];    % add future event
```

```
            end
```

```
        case DEPARTURE
```

```
            STATE= STATE - 1;
```

```
            TRANSMITTED= TRANSMITTED + 1;
```

```
    end
```

```
end
```

```
end
```

Exemplo 1: Simulador sem estimação de parâmetros de desempenho

```
function PB = VideoStreamingSimulator(lambda, invmiu, B, M, N)
```

```
...
```

```
% Initialization of variables and List of Events:
```

```
Clock= 0;           % simulation clock  
STATE= 0;           % no. of movies in transmission  
TRANSMITTED= 0;     % no. of transmitted movies  
N_ARRIVALS= 0;      % no. of movie requests  
BLOCKED= 0;        % no. of refused movies  
EventList= [ARRIVAL, Clock + exprnd(1/lambda)];
```

```
while TRANSMITTED < N
```

```
...
```

```
switch event
```

```
case ARRIVAL
```

```
EventList= [EventList; ARRIVAL, Clock + exprnd(1/lambda)]; % add future event
```

```
N_ARRIVALS= N_ARRIVALS + 1;
```

```
if STATE < M
```

```
STATE= STATE + 1;
```

```
EventList= [EventList; DEPARTURE, Clock + exprnd(invmiu)]; % add future event
```

```
else
```

```
BLOCKED= BLOCKED + 1;
```

```
end
```

```
case DEPARTURE
```

```
...
```

```
end
```

```
end
```

```
PB= BLOCKED / N_ARRIVALS;
```

```
end
```

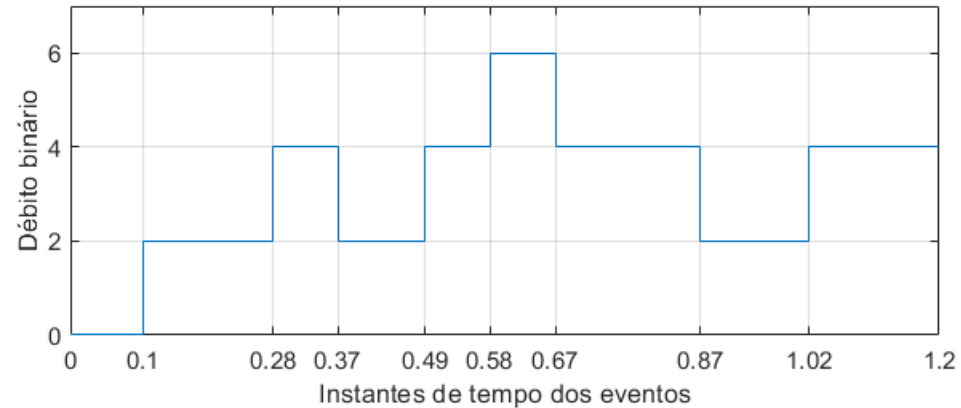
Exemplo 1: Simulador com estimaco da probabilidade de bloqueio

Exemplo 1:

Simulação de um servidor de *video-streaming*

Como calcular o contador estatístico LOAD (i.e., o integral do débito binário transmitido pelo servidor)

Ilustração da evolução do débito binário de uma simulação ($B = 2$ Mbps):



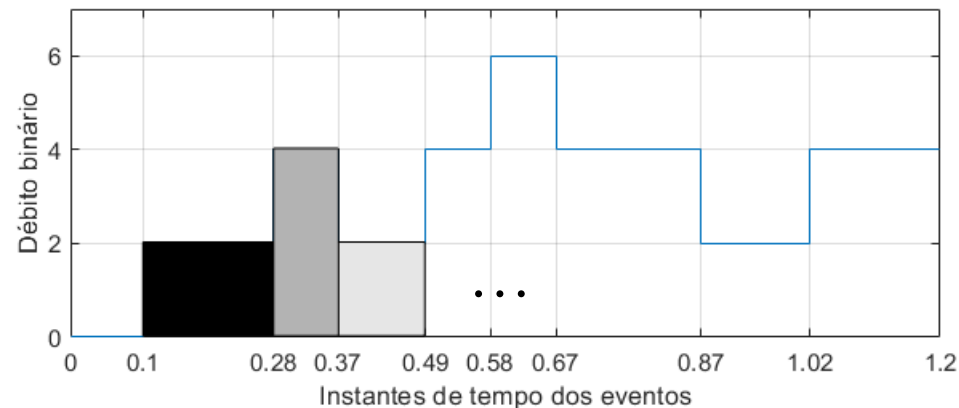
LOAD é a soma de retângulos.

Cada retângulo é:

diferença de tempo entre o relógio atual e o relógio do evento anterior

$\times$

o débito binário anterior



```
function [PB,DB] = VideoStreamingSimulator(lambda, invmiu, B, M, N)
```

```
...
```

```
% Initialization of variables and List of Events:
```

```
Clock= 0;           % simulation clock  
STATE= 0;           % no. of movies in transmission  
TRANSMITTED= 0;     % no. of transmitted movies  
N_ARRIVALS= 0;      % no. of movie requests  
BLOCKED= 0;         % no. of refused movies  
LOAD= 0;           % integral of the bitrate in transmission
```

```
EventList= [ARRIVAL, Clock + exprnd(1/lambda)];
```

```
while TRANSMITTED < N
```

```
    EventList= sortrows(EventList,2);    % sort EventList by time  
    event= EventList(1,1);               % register event type in first row  
    PreviousClock= Clock;                % save previous clock  
    Clock= EventList(1,2);               % update current clock with time instant of first row  
    EventList(1,:)= [];                  % delete first row of EventList  
    LOAD= LOAD + B*STATE*(Clock-PreviousClock);
```

```
    switch event
```

```
        case ARRIVAL
```

```
            ...
```

```
        case DEPARTURE
```

```
            ...
```

```
    end
```

```
end
```

```
PB= BLOCKED / N_ARRIVALS;
```

```
DB= LOAD/Clock;
```

```
end
```

Exemplo 1: Simulador com estimação do débito binário médio do servidor

Exemplo 1:

Simulação de um servidor de *video-streaming*

Anteriormente, foi assumido que os filmes têm uma duração exponencialmente distribuída com média de $1/\mu$ (em minutos)

A simulação pode ser mais realista se forem usados dados sobre os filmes. Considere-se que temos os seguintes dados:

Movie Title	Year	Length (min)	Av. Rating (0-5)
All About Eve	1950	79	1.7804
Annie Get Your Gun	1950	81	0.2153
Born Yesterday	1950	87	1.8627
Broken Arrow	1950	60	2.682
...	...	...	...
Toys	1992	100	3.7019
Twin Peaks: Fire Walk with Me	1992	107	1.5967
...	...	...	...
The Man Who Killed Don Quixote	2018	113	2.9703
Tom Segura: Disgraceful	2018	107	2.5626
Tomb Raider	2018	106	2.4702
When We First Met	2018	122	2.6908
Won't You Be My Neighbor?	2018	112	4.9569

Exemplo 1:

Simulação de um servidor de *video-streaming*

Assumindo que os filmes são pedidos com uma probabilidade proporcional à sua classificação (*Av. Rating*), podemos ter uma melhor caracterização estatística da duração dos filmes com os dados da tabela anterior.

- Seja F o conjunto dos filmes em que cada filme $f \in F$ é caracterizado pela sua duração d_f e pela sua classificação c_f (dados da tabela anterior).
- Seja D o conjunto dos valores de duração únicos dos filmes em F . Para cada duração $d \in D$, calcula-se o peso w_d com a soma das classificações c_f dos filmes $f \in F$ cuja duração é $d_f = d$.
- A probabilidade $p(d)$ de cada duração $d \in D$ é dada pelo seu peso w_d a dividir pela soma dos pesos de todas as durações.

Assim, a duração dos filmes é caracterizada por uma variável aleatória discreta em que D é o conjunto dos valores possíveis e $p(d)$ é a probabilidade de cada valor $d \in D$.

Exemplo 1:

Simulação de um servidor de *video-streaming*

Tabela completa em ficheiro CSV: moviesMPECI2025.csv

- Nomes e anos dos filmes reais
- Valores de duração e de classificação artificiais

Código MATLAB:

```
movies= readcell('moviesMPECI2025.csv','Delimiter',';');  
movies= movies(2:end,:);  
nMovies= length(movies);
```

Ler o ficheiro para
um *cell array*

```
durationTimes= zeros(1,nMovies);  
ratings= zeros(1,nMovies);  
for i = 1:nMovies  
    durationTimes(i)= movies{i,3};  
    ratings(i)= movies{i,4};  
end
```

Gerar:

durationTimes – vetor com a duração
de cada filme

ratings – vetor com a classificação
de cada filme

```
D= unique(durationTimes);  
w= zeros(1,length(D));  
for i= 1:length(D)  
    f= durationTimes == D(i);  
    w(i)= sum(ratings(f));  
end  
Prob= w/sum(w);
```

Gerar:

D – vetor com os valores únicos
de duração de filmes

Prob – vetor com as respetivas
probabilidades

Exemplo 1:

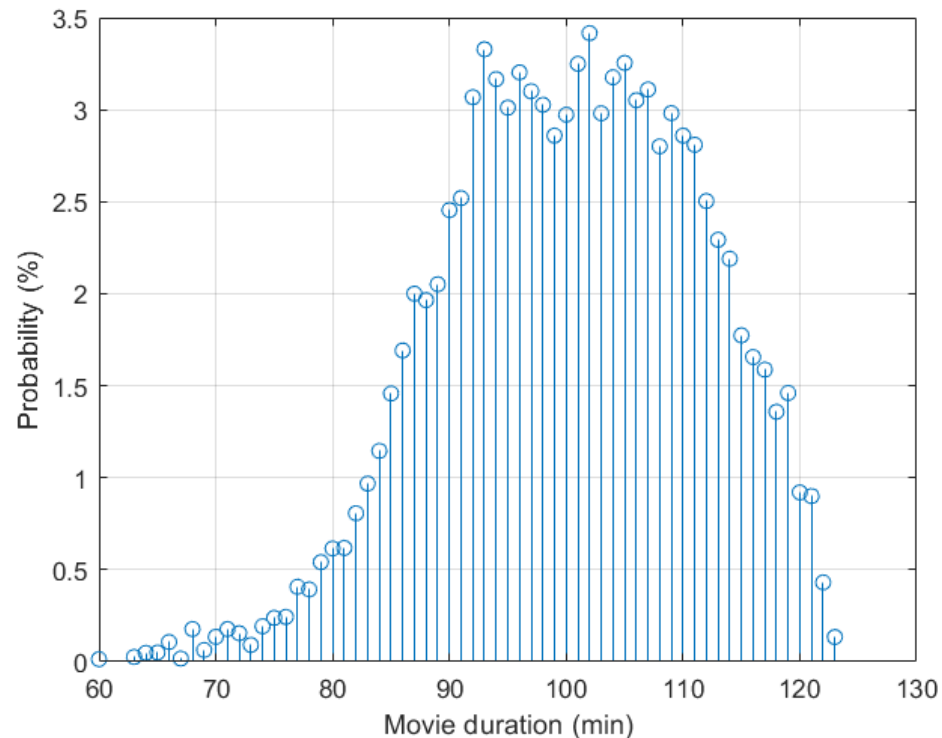
Simulação de um servidor de *video-streaming*

Tabela completa em ficheiro CSV: moviesMPECI2025.csv

- Nomes e anos dos filmes reais
- Valores de duração e de classificação artificiais

Duração média dos filmes: 100.39 minutos

Função Massa de
Probabilidade:



Resultados de simulação de um servidor de *video-streaming*

Um servidor de *video-streaming* caracterizado por:

- os pedidos de filmes a uma taxa de $\lambda = 2$ pedidos por minuto
- cada filme é transmitido a um débito de $B = 2$ Mbps
- o servidor tem uma capacidade para servir até $M = 200$ filmes

Assumindo a duração dos filmes exponencial de média $1/\mu = 100.39$ min, resultados de 2 simulações (critério de paragem $N = 10^6$)

Blocking probability = 5.81%

Avg. server throughput = 378.97 Mbps

Blocking probability = 5.86%

Avg. server throughput = 379.18 Mbps

Assumindo a duração dos filmes com a distribuição discreta determinada anteriormente, resultados de 2 simulações (critério de paragem $N = 10^6$)

Blocking probability = 5.64%

Avg. server throughput = 378.73 Mbps

Blocking probability = 5.75%

Avg. server throughput = 378.92 Mbps

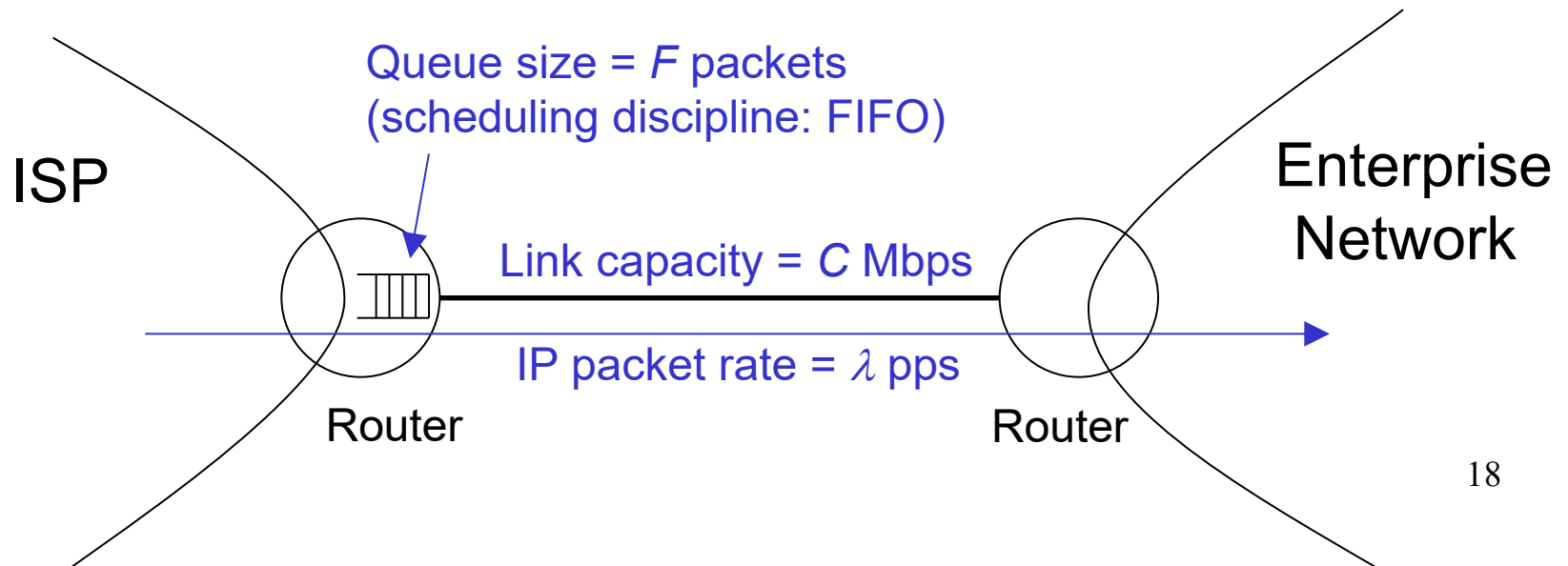
CONCLUSÃO: Considerar filmes com uma duração exponencial é uma boa aproximação desde que a média seja real.

Exemplo 2:

Ligação de dados de um ISP para uma empresa

Considere-se o sentido *downstream* de uma ligação entre a rede de uma empresa e o seu ISP (*Internet Service Provider*) caracterizado por:

- ligação com capacidade de C Mbps
- chegada de pacotes ao router do ISP com destino para a empresa segundo um processo de Poisson a uma taxa de λ pacotes/segundo
- fila de espera com capacidade de F pacotes
- pacotes com tamanho exponencial de média B bytes

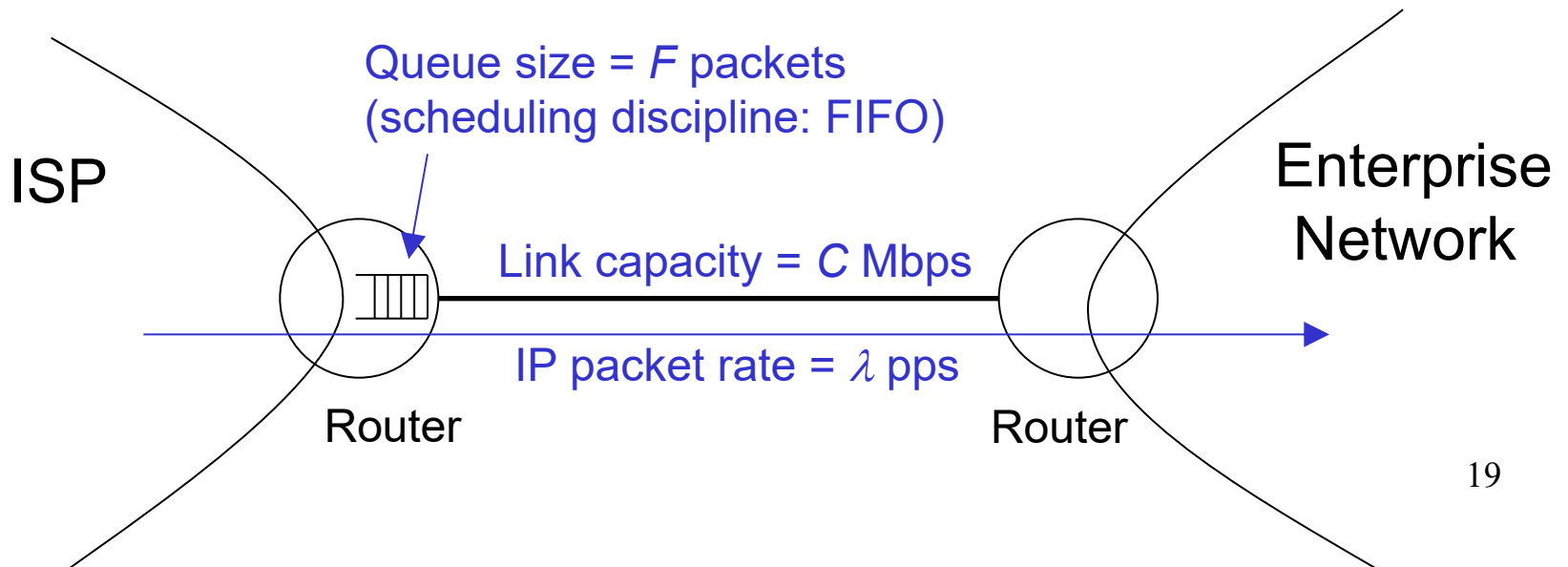


Exemplo 2:

Ligação de dados de um ISP para uma empresa

Pretende-se estimar:

- o atraso médio por pacote, i.e., o tempo médio desde que cada pacote chega ao router do ISP até que acaba de ser transmitido pela ligação.



Exemplo 2:

Ligação de dados de um ISP para uma empresa

Eventos:

ARRIVAL – chegada de um pacote

DEPARTURE – fim da transmissão de um pacote

Variáveis de estado:

STATE – variável binária que é 0 (se a ligação está livre) ou 1 (se a ligação está ocupada a transmitir um pacote)

QUEUE_N – número de pacotes na fila de espera

Critério de paragem:

N – a simulação termina no fim da transmissão do Nésimo pacote *

* neste caso, como vamos ver no slide seguinte, a variável auxiliar TRANSMITTED também é um contador estatístico

Exemplo 2:

Ligação de dados de um ISP para uma empresa

Contadores estatísticos necessários:

DELAYS – soma dos atrasos dos pacotes transmitidos até ao instante presente (cada atraso é a diferença do instante chegada do pacote até ao instante de fim de transmissão do pacote)

TRANSMITTED – número de pacotes transmitidos até ao instante presente

Parâmetro de desempenho de interesse:

Atraso médio por pacote (AM):

$$AM = DELAYS / TRANSMITTED$$

Variável auxiliar necessária:

QUEUE – vetor com os instantes de chegada dos pacotes em fila de espera

Exemplo 2: Ligação de dados de um ISP para uma empresa

```
function AM = LinkSimulator(lambda,B,C,F,N)
    %Events:
    ARRIVAL= 0;      % Arrival of a packet
    DEPARTURE= 1;    % Departure of a packet
    % Initialization of variables:
    Clock= 0;        % Simulation clock
    STATE = 0;       % 0 - connection is free; 1 - connection is occupied
    QUEUE_N= 0;      % No. of packets in queue
    DELAYS= 0;       % Sum of the delays of transmitted packets
    TRANSMITTED= 0;  % N. of transmitted packets
    QUEUE= [];       % Arriving time instant of each packet in the queue
    ...
```

- Definição dos tipos de eventos
- Inicialização de todas as variáveis

Exemplo 2: Ligação de dados de um ISP para uma empresa

```
function AM = LinkSimulator(lambda,B,C,F,N)
```

```
...
```

```
% Initializing the List of Events with the first ARRIVAL:
```

```
tmp= Clock + exprnd(1/lambda);
```

```
EventList = [ARRIVAL, tmp, tmp];
```

Geração de um instante de tempo
exponencialmente distribuído para
o primeiro ARRIVAL

```
while TRANSMITTED < N
```

```
EventList= sortrows(EventList,2); % sort EventList by time
```

```
Event= EventList(1,1); % register Event type in first row
```

```
Clock= EventList(1,2); % update Clock with time instant of first row
```

```
ArrInstant= EventList(1,3); % register arrival instant of packet
```

```
EventList(1,:)= []; % delete first row of EventList
```

```
...
```

EventList:

	Tipo de evento	Instante do evento	Instante de chegada do pacote	
	↓	↓	↓	
	ARRIVAL	0.23	0.23	← Os valores são iguais
	DEPARTURE	0.54	0.23	← Atraso deste pacote = $0.54 - 0.23$
	ARRIVAL	0.56	0.56	
	...	...	...	

Exemplo 2: Ligação de dados de um ISP para uma empresa

```
function AM = LinkSimulator(lambda,B,C,F,N)
```

```
...
```

```
while TRANSMITTED < N
```

```
...
```

```
switch Event
```

```
case ARRIVAL
```

```
tmp= Clock + exprnd(1/lambda);
```

```
EventList = [EventList; ARRIVAL, tmp, tmp];
```

```
if STATE == 0
```

```
STATE= 1;
```

```
tTime= 8*exprnd(B)/(C*1e6); % in seconds
```

```
EventList = [EventList; DEPARTURE, Clock + tTime, ArrInstant];
```

```
else
```

```
if QUEUE_N < F
```

```
QUEUE_N= QUEUE_N + 1;
```

```
QUEUE= [QUEUE; ArrInstant];
```

```
end
```

```
end
```

```
...
```

Agendamento do
próximo ARRIVAL

Geração de um tamanho
exponencialmente distribuído
para o pacote

Se ligação livre, o pacote
começa a ser transmitido e
é agendado o seu
DEPARTURE

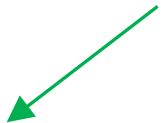
Se ligação ocupada e o pacote cabe na
fila de espera, o seu instante de
chegada é colocado no fim de QUEUE

- Processamento dos eventos do tipo
ARRIVAL (chegada de um pacote)

Exemplo 2: Ligação de dados de um ISP para uma empresa

```
function AM = LinkSimulator(lambda,B,C,F,N)
...
while TRANSMITTED < N
...
switch Event
case ARRIVAL
...
case DEPARTURE
    DELAYS= DELAYS + (Clock - ArrInstant);
    TRANSMITTED= TRANSMITTED + 1;
    if QUEUE_N > 0
        QInstant= QUEUE(1);
        tTime= 8*exprnd(B)/(C*1e6); % in seconds
        EventList = [EventList; DEPARTURE, Clock + tTime, QInstant];
        QUEUE_N= QUEUE_N - 1;
        QUEUE(1)= [];
    else
        STATE= 0;
    end
end
end
AM= DELAYS/TRANSMITTED;
end
```

Atualização dos contadores estatísticos



Quando um pacote sai da fila de espera para ser transmitido, o seu instante de chegada é colocado no DEPARTURE da lista de eventos



- Processamento dos eventos do tipo DEPARTURE (fim da transmissão de um pacote)
- Determinação final do AM

Geração de valores aleatórios

- Um simulador de eventos discretos exige a geração de valores aleatórios com determinadas distribuições.
 - Nas simulações anteriores, por exemplo, foi preciso gerar valores aleatórios com uma distribuição exponencial para os intervalos entre pedidos de filmes (exemplo 1) e para os intervalos entre chegadas de pacotes (exemplo 2)
- Muitos dos métodos computacionais de geração de valores aleatórios baseiam-se em:
 1. Gerar valores aleatórios reais com uma distribuição uniforme entre 0 e 1
 2. Transformar os valores por forma a seguirem uma determinada distribuição de interesse.
- Assim, o primeiro passo é perceber como se geram valores aleatórios reais com uma distribuição uniforme entre 0 e 1.

Geração de valores aleatórios reais com uma distribuição uniforme entre 0 e 1

Algoritmos Congruenciais Lineares

- Os métodos mais comuns para gerar sequência pseudo-aleatórias usam os chamados geradores congruências lineares LCG (*linear congruencial generators*)
- Este geradores geram uma sequência de números através da fórmula recursiva:

$$X_{i+1} = (aX_i + c) \mod m$$

em que X_0 é a “semente” (*seed*) e a , c , m (inteiros não-negativos) são designados de multiplicador, incremento e módulo, respetivamente .

- Quando $c = 0$ o gerador designa-se por congruencial multiplicativo.

Geração de valores aleatórios reais com uma distribuição uniforme entre 0 e 1

Algoritmos Congruenciais Lineares

$$X_{i+1} = (aX_i + c) \bmod m$$

- Como X_i pode apenas assumir os valores $\{0, 1, \dots, m-1\}$, os números

$$U_i = \frac{X_i}{m}$$

são designados por números pseudo-aleatórios e constituem uma aproximação a uma sequência de variáveis aleatórias uniformemente distribuídas entre 0 e 1

Algoritmos Congruenciais Lineares (Exemplo)

$$Z_i = (5Z_{i-1} + 3) \bmod 16$$

$$Z_0 = 7$$

$$Z_1 = (5 \times 7 + 3) \bmod 16$$

$$= 38 \bmod 16 = 6$$

$$U_1 = 6 / 16 = 0.375$$

i	Z_i	U_i	i	Z_i	U_i
0	7	----	10	9	0.563
1	6	0.375	11	0	0.000
2	1	0.063	12	3	0.188
3	8	0.500	13	2	0.125
4	11	0.688	14	13	0.813
5	10	0.625	15	4	0.250
6	5	0.313	16	7	0.438
7	12	0.750	17	6	0.375
8	15	0.938	18	1	0.063
9	14	0.875	19	8	0.500

- Neste exemplo, $m = 16$ e o gerador repete os valores gerados ao fim de 16 iterações (diz-se que o gerador tem um período de 16).

Geração de valores aleatórios reais com uma distribuição uniforme entre 0 e 1

Algoritmos Congruenciais Lineares

$$X_{i+1} = (aX_i + c) \bmod m$$

- Como temos um total de m valores possíveis, o ciclo de repetição da sequência é no máximo m pelo que o período do gerador também é no máximo m
- Mas pode ser muito menor
 - Exemplo: $a = c = X_0 = 3$ e $m = 5$ gera a sequência $\{3, 2, 4, 0, 3 \dots\}$ com período 4
- Apenas algumas combinações de parâmetros produzem resultados satisfatórios
 - Exemplo: Usar $m = 2^{31} - 1$, $a = 7^5$ e $c = 0$ em computadores de 32 bits

Função Matlab *rand()*

A função *rand()* do Matlab gera valores aleatórios reais com distribuição uniforme entre 0 e 1.

A função permite escolher a semente do gerador. Escolhendo a mesma semente, o gerador gera a mesma sequência de valores.

```
>> rand('state',12)
>> a= rand()
a =
    0.5266
>> b= rand(1,4)
b =
    0.7502    0.6655    0.9640    0.1082
>> rand('state',12)
>> a= rand()
a =
    0.5266
>> a= rand()
a =
    0.7502
```

Escolha da semente 12

Sequência de valores gerados

Escolha novamente da semente 12

Mesma sequência de valores gerados

Geração de valores de variáveis aleatórias discretas

Pretende-se gerar um valor aleatório em que:

- os valores possíveis são $X_1, X_2, \dots, X_n$
- a probabilidade de cada valor é $P(X = X_i) = f_i$

Método:

- Dividir o intervalo $[0,1]$ em n intervalos proporcionais a $f_i, i = 1 \dots n$
- Gerar um valor aleatório real U com uma distribuição Uniform(0,1)
- Retornar X_i se U estiver no i ésimo intervalo

Por exemplo, um valor aleatório U de uma variável de Bernoulli $X = \{0,1\}$ com $p(0) = 1/4$ e $p(1) = 3/4$ pode ser gerado da seguinte forma:

(1) Gerar $U \sim \text{Uniform}(0,1)$

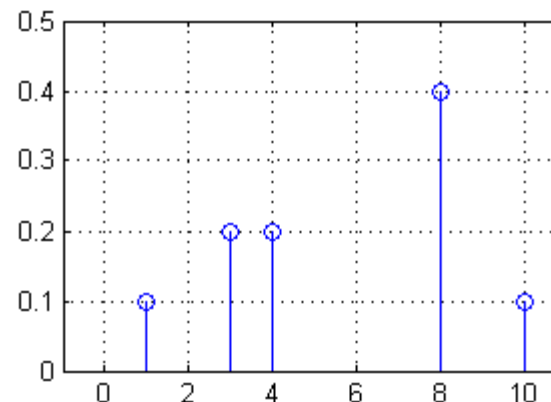
(2) Se $U \leq 1/4$, retornar $X = 0$; senão, retornar $X = 1$

Geração de valores de variáveis aleatórias discretas

Variáveis discretas (exemplo Matlab)

```
x= [1 3 4 8 10];  
f= [0.1 0.2 0.2 0.4 0.1];
```

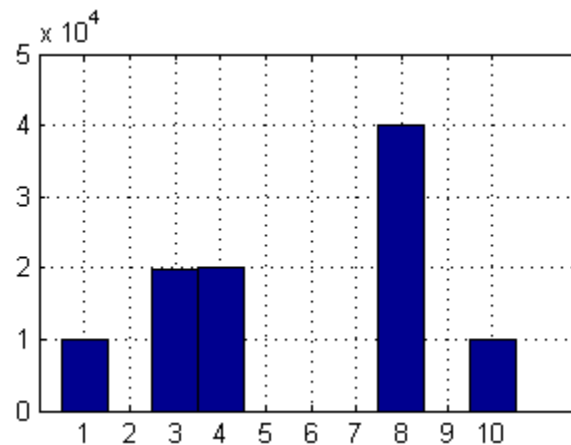
```
figure(1)  
stem(x,f)  
axis([-1 11 0 0.5])  
grid on
```



```
f_cum= [0 cumsum(f)]  
  
a= zeros(1,100000);  
for it= 1:100000  
    a(it)= x(sum(rand()>f_cum));  
end
```

```
f_cum =  
  
0.0  0.1  0.3  0.5  0.9  1.0
```

```
figure(2)  
histogram(a,1:10)  
grid on
```



Geração de valores de variáveis aleatórias contínuas

Distribuição uniforme entre a e b , com $a < b$

Método:

- Gerar um valor aleatório real U com uma distribuição Uniform(0,1)
- Retornar $X = (b - a) \times U + a$

Exemplos:

- $X = 2U + 1$ gera um valor real uniforme no intervalo $[1, 3]$
- $X = 3U - 1.5$ gera um valor real uniforme no intervalo $[-1.5, 1.5]$

Geração de valores de variáveis aleatórias contínuas

Método baseado na função de distribuição acumulada $F(X)$ de uma variável aleatória contínua X

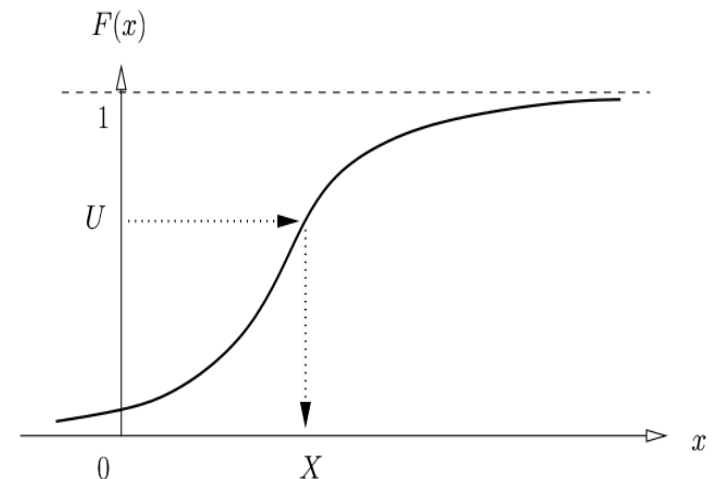
Considerando:

- $F^{-1}(X)$ a inversa da função de distribuição acumulada
- a variável aleatório U com distribuição uniforme entre 0 e 1

prova-se que a variável aleatória $X = F^{-1}(U)$ tem como função de distribuição acumulada $F(X)$.

Assim, quando $F^{-1}(X)$ é conhecida, podemos usar o método:

- Gerar um valor aleatório real U com uma distribuição Uniform(0,1)
- Retornar $X = F^{-1}(U)$



Geração de valores de variáveis aleatórias contínuas

Método baseado na função de distribuição acumulada $F(X)$ de uma variável aleatória contínua X

Distribuição exponencial com taxa λ

$$F(x) = \begin{cases} 1 - e^{-\lambda x}, & x \geq 0 \\ 0, & x < 0 \end{cases} \quad F^{-1}(U) = -\frac{1}{\lambda} \ln(U)$$

Método:

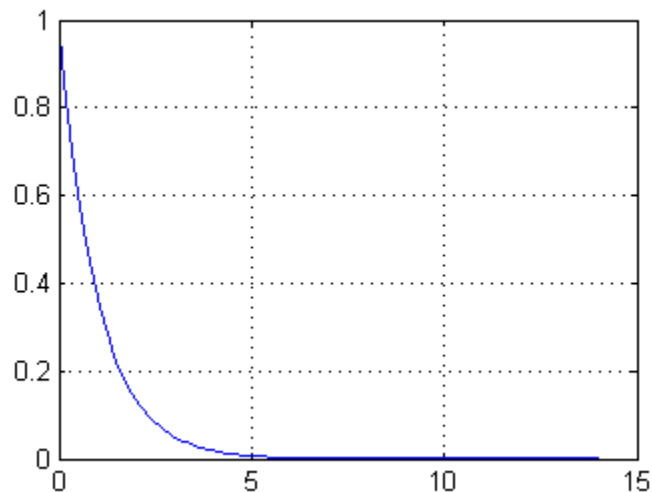
- Gerar um valor aleatório real U com uma distribuição Uniform(0,1)
- Retornar $X = -1/\lambda \times \ln(U)$

No Matlab, usa-se a função `exprnd()`

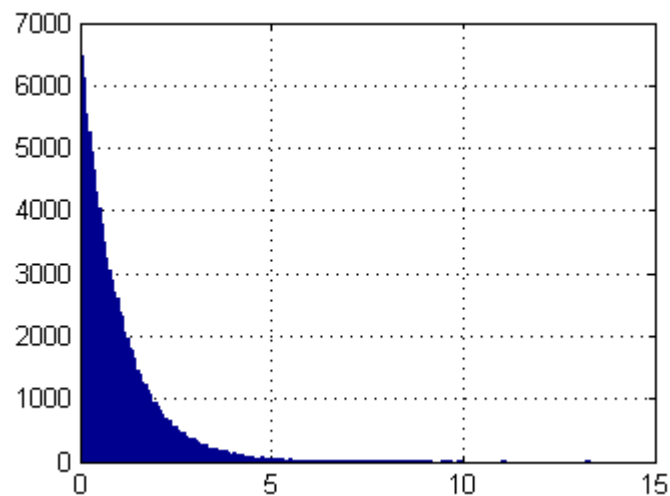
Geração de valores de variáveis aleatórias contínuas

Variável exponencial (em MATLAB)

```
lambda= 1  
x= 0:0.1:14;  
f=exppdf(x,1/lambda)  
figure(1)  
plot(x,f)  
grid on
```



```
a=exprnd(1/lambda,1,100000);  
figure(2)  
hist(a,200)  
grid on
```

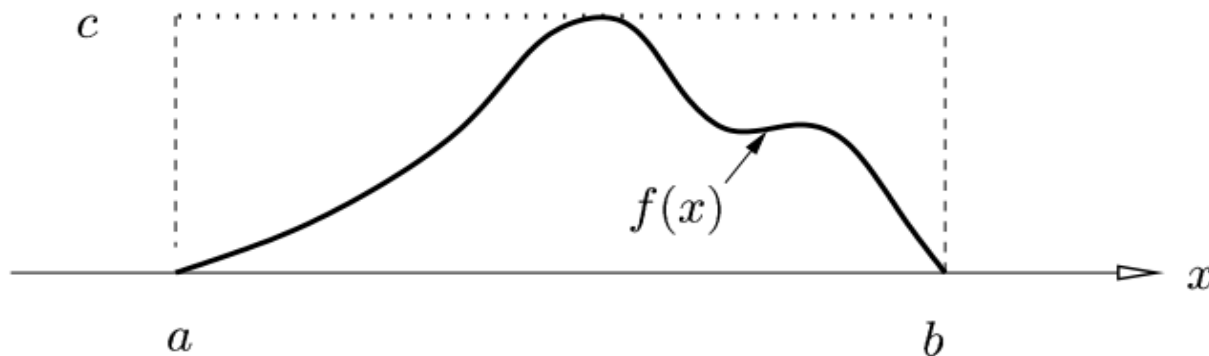


Geração de valores de variáveis aleatórias contínuas

Método baseado na rejeição

Se considerarmos uma variável aleatória x contínua com uma função densidade de probabilidade $f(x)$

- define-se uma área que contém todos os valores da função densidade de probabilidade no intervalo em que está definida
- geram-se números com distribuição uniforme na área definida e rejeitam-se os que ficam acima de $f(x)$

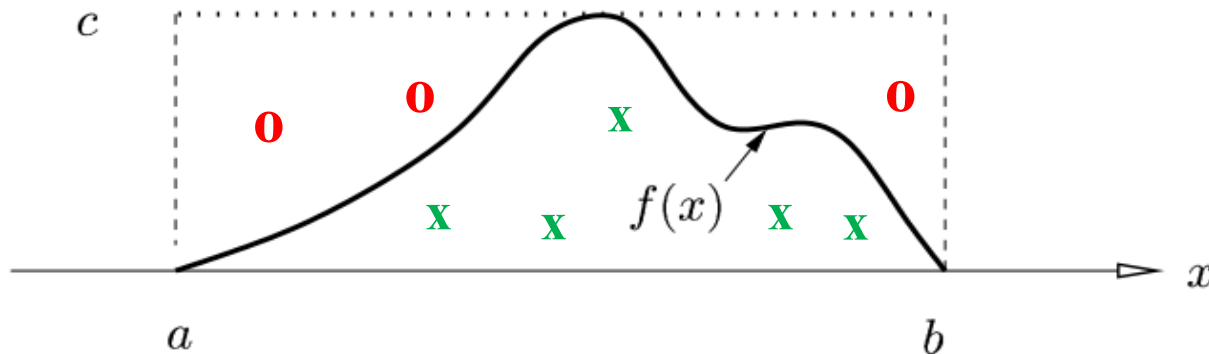


Geração de valores de variáveis aleatórias contínuas

Método baseado na rejeição

Método:

1. Gerar um valor aleatório real X com uma distribuição $\text{Uniform}(a,b)$
2. Gerar um valor aleatório real Y com uma distribuição $\text{Uniform}(0,c)$
3. Se $Y \leq f(X)$, retornar X ; caso contrário, voltar ao passo 1



Análise de resultados de simulação

- Considere-se os resultados de n simulações de um sistema representados pelas variáveis aleatórias $X_1, X_2, \dots, X_n$ para um determinado parâmetro de desempenho. Em geral, os n resultados são diferentes.
- Se $X_1, X_2, \dots, X_n$ forem variáveis independentes e identicamente distribuídas, cada uma com a mesma média μ e o mesmo desvio padrão σ , então:
 - a média amostral (soma das variáveis a dividir pelo no. de variáveis) tende para uma distribuição normal (pelo Teorema do Limite Central)
 - podemos apresentar os resultados na forma de um Intervalo de Confiança (relembrar o módulo sobre Intervalos de Confiança dado anteriormente e como se determina o IC quando o desvio padrão σ não é conhecido)
- O objetivo é apresentar um intervalo de valores para o qual temos uma confiança a $100 \times (1 - \alpha)\%$ que a média μ (que queremos estimar) está dentro do intervalo.

Exemplo 1: Simulação de um servidor de *video-streaming* em MATLAB

```
lambda= 2; % request/minute  
invmiu= 90; % minutes  
B= 2; % Mbps  
M= 200; % movies  
N= 1e4; % Simulation stop criterion
```

Parâmetros de entrada

```
Nsim = 20; % number of simulations  
PB= zeros(1,Nsim); % vector to save results  
DB= zeros(1,Nsim); % vector to save results  
for it= 1:Nsim  
    [PB(it), DB(it)]= VideoStreamingSimulator(lambda, invmiu, B, M, N);  
end
```

Correr o simulador *Nsim* vezes e guardar os resultados em *PB* e *DB*

```
alfa= 0.1; % 90% confidence interval  
media = mean(PB);  
[~, ~, ci] = ttest(PB, media, 'alpha', alfa);  
fprintf('PB = %.4f [%.4f - %.4f] \n', media, ci(1), ci(2));  
media = mean(DB);  
[~, ~, ci] = ttest(DB, media, 'alpha', alfa);  
fprintf('DB = %.2f Mbps [%.2f - %.2f] \n', media, ci(1), ci(2));
```

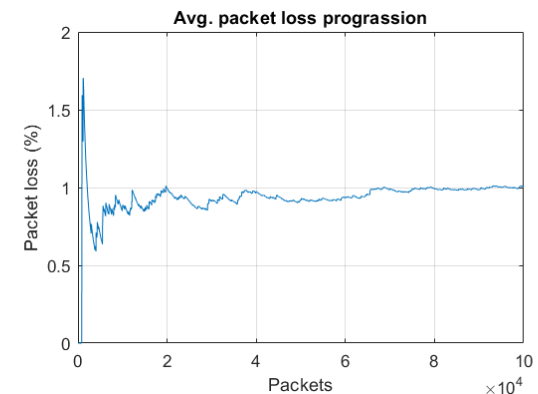
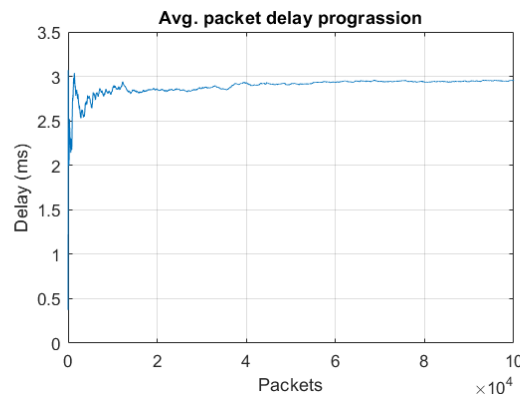
Calcular média e intervalo de confiança para cada parâmetro de desempenho

Análise de resultados de simulação

- O Teorema do Limite Central exige que as variáveis $X_1, X_2, \dots, X_n$ sejam independentes e identicamente distribuídas.
 - Uma forma de garantir a independência é correr as diferentes simulações garantindo que os valores gerados aleatoriamente são diferentes entre simulações.
 - Isto é conseguido usando diferentes sementes (*seeds*) nos geradores de valores aleatórios.
- O intervalo de confiança dá uma medida de quão próximo o valor estimado poderá estar do seu valor real:
 - Se o intervalo é pequeno, o valor estimado deverá estar próximo.
 - Se o intervalo é grande, não sabemos quão bom é o valor estimado.
- Para um determinado no. de simulações, se o intervalo de confiança não for suficientemente pequeno, ele pode ser tornado mais pequeno:
 - com um no. maior de simulações, e/ou
 - com simulações maiores (i.e., com maior tempo de simulação).

Análise de resultados de simulação

Em geral, um processo estocástico têm um regime transitório inicial (que depende do estado inicial do sistema simulado) antes de chegar ao regime estacionário.



- Para garantir que os parâmetros de desempenho estimados são corretos, pode ser necessário esperar um tempo de *warm-up* para que o estado inicial transitório se desvaneça.
 - Se o tempo simulado for muito maior que o tempo de *warm-up*, os contadores estatísticos podem ser inicializados no início da simulação.
 - Caso contrário, os contadores estatísticos devem ser inicializados apenas após o tempo de *warm-up* (tempo este que deve ser previamente estimado).